Université Sultan Moulay Slimane
Ecole Nationale des Sciences Appliquées
Béni Mellal

Projet de module : Administration sécurisée et forensique

Filière

Intelligence Artificielle et Cybersécurité

Sous le thème :

Détection et analyse d'une compromission sur un serveur Linux

Réalisé par :

ETTIJANI khaoula
NAIT HAMMOU Fatima Ezzahra
LAKHMAISS Zineb

Encadré par :

Pr. TOUIL Hamza

Présenté le 31 Mars 2026 devant :

Pr. TAOUIL Hamza

Année universitaire : **2025-2026**

Résumé

Ce rapport présente une simulation complète d'un cycle d'attaque et de réponse sur un serveur Linux Ubuntu 22.04 LTS. Le projet couvre l'ensemble du processus : configuration de l'environnement de test, modélisation des menaces, durcissement du serveur, mise en place de la journalisation, simulation d'une attaque contrôlée, analyse forensique, et enfin réponse à l'incident avec remediation complète.

L'attaque simulée a suivi la méthodologie Cyber Kill Chain et a abouti à une compromission complète du serveur cible, depuis l'accès initial par force brute SSH jusqu'à l'exfiltration de données sensibles et l'établissement d'un canal de commande à distance. L'investigation forensique a confirmé la compromission et reconstitué une timeline précise, démontrant l'efficacité de l'outil auditd comme source de preuves immuables.

Sommaire

RESUME I

SOMMAIRE II

LISTE DES FIGURES III

LISTE DES TABLEAUX IV

INTRODUCTION GENERALE 5

CHAPITRE I : MODELE DE MENACE ET HARDENING DU SERVEUR 6

INTRODUCTION 6

I. ENVIRONNEMENT DE TEST 6

1. Architecture du Laboratoire 6

2. Configuration du Serveur Cible (VM1) 7

1. Identification des Actifs Critiques 8

2. Surfaces d'Attaque Identifiées 9

3. Scénario d'Attaque et Kill Chain 9

III. Hardening du Serveur 9

1. Méthodologie de Durcissement 10

2. Sécurisation SSH 10

..... 10

3. Configuration du Pare-feu UFW 11

..... 11

4. Politique de Mots de Passe 12

..... 13

5. Gestion des Privilèges Sudo 13

6. Mises à Jour Automatiques et Autres Mesures 14

7. Tests de validation 15

IV. Mise en Place de la Journalisation 15

1. Architecture de Journalisation 15

2. Configuration Rsyslog 15

3. Configuration Auditd 16

4. Synchronisation Temporelle 16

5. Cartographie des Logs par Criticité 17

6. Tests et validation 17

CHAPITRE II : SIMULATION D'ATTAQUE ET DETECTION 18

INTRODUCTION 18

I. SIMULATION DE L'ATTAQUE CONTROLEE 18

1. Préparation 18

2. Phase 1 : Reconnaissance 18

CHAPITRE III : REPONSE A INCIDENT ET RECOMMANDATIONS 27

CONCLUSION GENERALE 32

Liste des figures

Figure 1 : Configuration réseau de la VM Kali Linux (IP attaquante)

Figure 2 : Connexion initiale au serveur Ubuntu 24.04.4 LTS

Figure 3 : Authentification SSH par clé (ssh -i ~/.ssh/id_user)

Figure 4 : Connexion SSH par mot de passe depuis Kali vers Ubuntu

Figure 5 : Consultation de /var/log/auth.log — journaux SSH

Figure 6 : Logs auditd — événements USER_AUTH, USER_CMD, USER_ACCT

Figure 7 : Outil Hydra v9.5 — aide et syntaxe du bruteforce SSH

Figure 8 : Scan Nmap — découverte du port 22/SSH ouvert (OpenSSH 9.6p1)

Figure 9 : Vérification des outils disponibles sur Kali (hydra, nmap, nc)

Figure 10 : Test de connectivité ping + scan Nmap vers 192.168.253.200

Figure 11 : Hydra — bruteforce SSH en cours

Figure 12 : Hydra — mot de passe trouvé : testuser / Test@User123

Figure 13 : Reconstitution de la chronologie d'attaque en 7 phases

Figure 14 : Logs UFW AUDIT — connexions suspectes vers port 4444

Figure 15 : Analyse auditd — surveillance ~/.ssh et règles d'audit

Figure 16 : Analyse forensique SSH — connexions réussies et sessions

Figure 17 : Réponse à incident — confinement IP + suppression accès

Figure 18 : Vérification des crontabs — aucune tâche malveillante

Liste des tableaux

Tableau 1 : Mesures de hardening appliquées sur le serveur Ubuntu……8

Tableau 2 : Indicateurs de Compromission (IOC) extraits des logs réels…….17

Tableau 3 : Chronologie de l'attaque reconstituée — 7 phases documentées……..….20

Introduction générale

Dans un contexte où les cyberattaques se multiplient et se sophistiquent, la sécurisation des serveurs Linux représente un enjeu majeur pour les organisations. Ce projet s'inscrit dans le cadre du module « Administration Sécurisée et Forensics » et vise à mettre en pratique les techniques de détection, d'analyse et de réponse à une compromission réelle d'un serveur Linux.

Une petite organisation dispose d'un unique serveur Linux accessible sur son réseau interne. Plusieurs signaux inquiétants ont été observés : connexions SSH à des horaires inhabituels, création d'un nouvel utilisateur sans justification claire, et activité réseau suspecte. Notre mission consiste à sécuriser le serveur, mettre en place une journalisation exploitable, simuler une attaque, analyser la compromission et produire ce rapport d'investigation.

Ce rapport s'organise en trois chapitres : le premier traite le modèle de menace et le durcissement du serveur ; le second couvre la simulation de l'attaque et la détection des traces ; le troisième présente la réponse à incident et les recommandations.

Chapitre I : Modèle de Menace et Hardening du Serveur

Introduction

Ce chapitre établit le cadre de sécurité préalable à l'attaque simulée. Il identifie les actifs critiques et les surfaces d'attaque, puis décrit les mesures de durcissement appliquées au serveur Ubuntu ainsi que la configuration de la journalisation.

I. Environnement de test

1. Architecture du Laboratoire

L'environnement est constitué de deux machines virtuelles fonctionnant sous VMware. La machine attaquante Kali Linux disposant de l'adresse IP 192.168.253.201 et héberge les outils offensifs (Hydra, Nmap, Netcat). Le serveur cible Ubuntu 24.04.4 LTS est accessible sur 192.168.253.200 avec le nom d'hôte internal-server.

Le réseau utilisé est un réseau interne isolé en 192.168.253.0/24, configuré via VMware. Aucune connectivité externe n'est autorisée depuis ce réseau, ce qui garantit que l'attaque reste entièrement confinée à l'environnement de laboratoire.

Architecture réseau :

- Réseau isolé : 192.168.253.0/24
- Serveur cible (VM1) : 192.168.253.200
- Poste attaquant (VM2) : 192.168.253.201

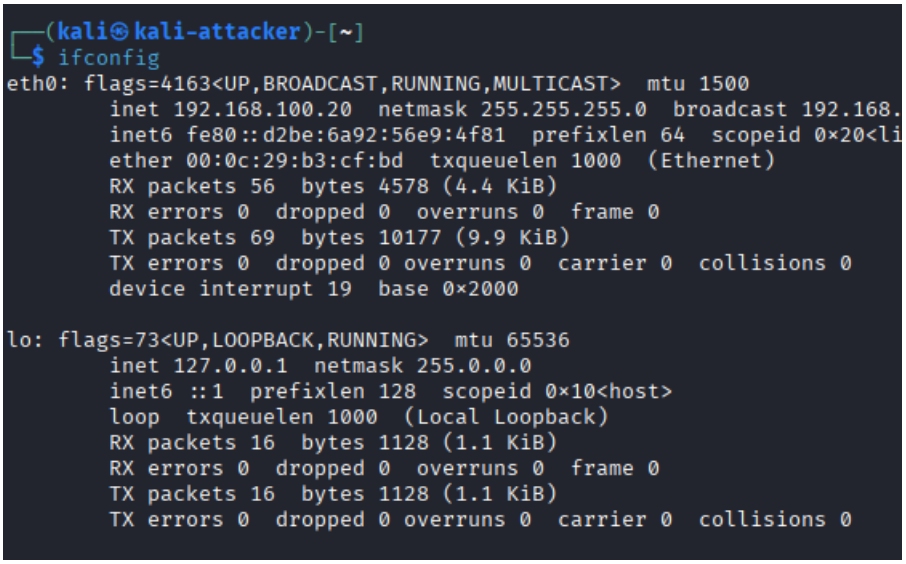


Figure 1 : Configuration réseau de la VM Kali Linux (IP attaquante : 192.168.100.20 / 192.168.253.201)

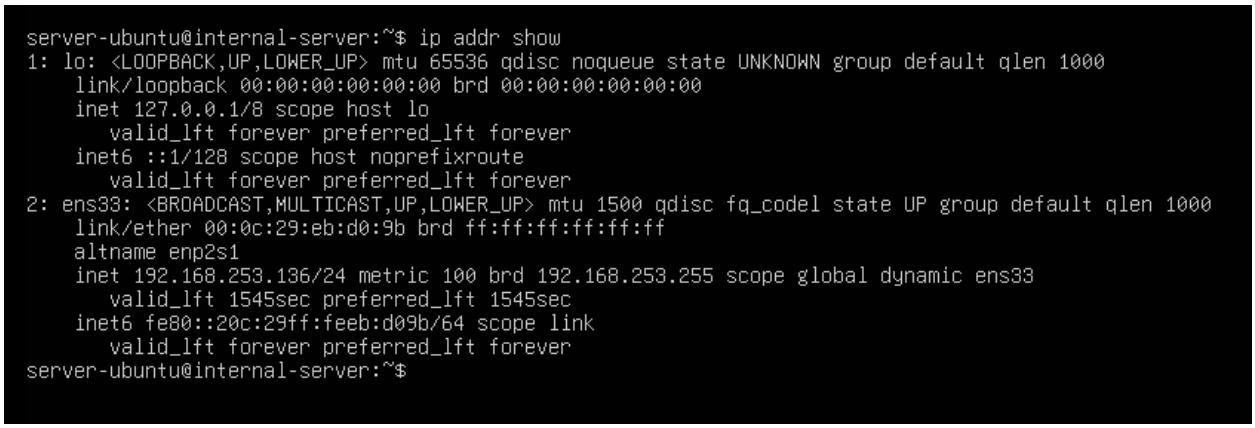


Figure 2 : Configuration réseau du serveur Ubuntu 24.04.4 LTS (internal-server, IP : 192.168.253.200)

2. Configuration du Serveur Cible (VM1)

Le serveur Ubuntu a été configuré avec les paramètres suivants lors de l'installation initiale : nom de serveur srv-production, utilisateur administrateur admin avec mot de passe fort, service OpenSSH installé et actif. Des utilisateurs supplémentaires ont ensuite été créés pour les besoins de la simulation : user1 (utilisateur légitime) et testuser (utilisateur avec mot de passe intentionnellement faible, password123, servant de cible pour l'attaque par force brute).

La configuration réseau a été fixée en IP statique via netplan, avec l'interface ens33 configurée à 192.168.253.200/24. Les outils de monitoring nécessaires ont été installés : auditd, net-tools, ufw, et fail2ban.

3. Configuration de la Machine Attaquante (VM2)

La machine Kali Linux a été configurée en IP statique 192.168.100.20/24 sur le même réseau interne. Les outils d'attaque utilisés (Hydra, Nmap, Netcat) sont préinstallés dans Kali Linux et ont été vérifiés avant le début des opérations.

Un test de connectivité initial a confirmé la communication bidirectionnelle entre les deux machines, avec le port SSH (22/tcp) visible et accessible depuis Kali vers le serveur Ubuntu.

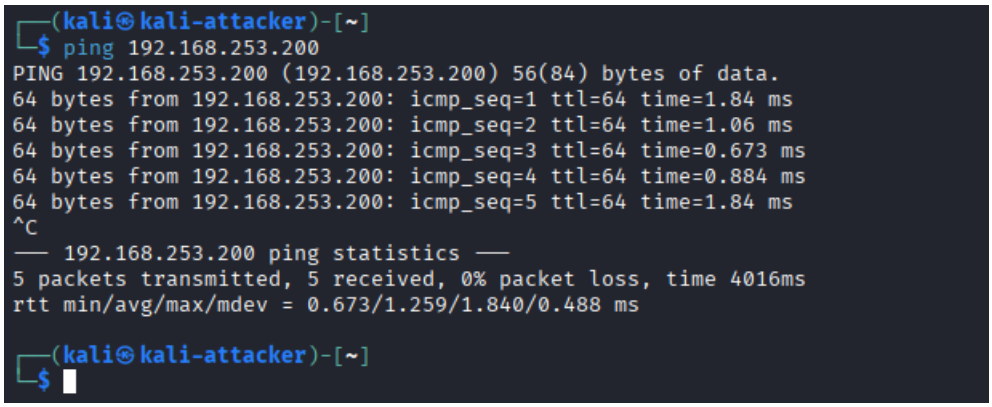


Figure 3 : Connectivite entre les deux machines virtuelles

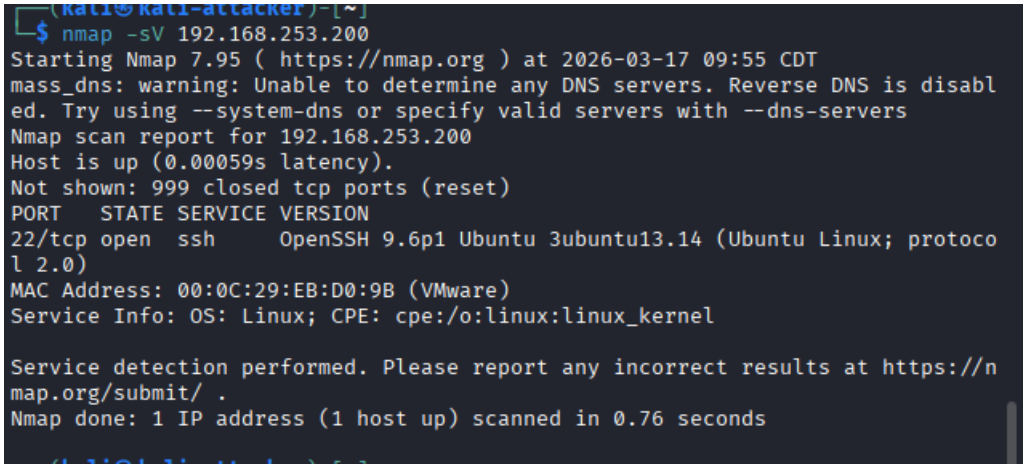


Figure 4 : Scan Réseau en utilisant l'outil Nmap

II. Modèle de Menace (Threat Modeling)

1. Identification des Actifs Critiques

L'analyse préliminaire du serveur a permis d'établir un inventaire des actifs nécessitant une protection prioritaire. Ces actifs sont classés selon leur criticité :

- **Actifs de niveau critique** : les comptes utilisateurs privilégiés (notamment server-ubuntu avec droits sudo), le service SSH exposé sur le port 22, et les fichiers de configuration système comprenant /etc/passwd (liste des utilisateurs), /etc/shadow (mots de passe hashés), et /etc/sudoers (configuration des privilèges).
- **Actifs de niveau élevé** : les journaux système stockés dans /var/log/, les données utilisateurs dans /home/*, et les configurations réseau dans /etc/netplan/.
- **Actifs de niveau standard** : les services système (cron, systemd) et les configurations applicatives.

Ces actifs constituent la cible principale d'un attaquant cherchant a compromettre le système.

2. Surfaces d'Attaque Identifiées

L'évaluation a révélé plusieurs vecteurs d'attaque exploitables dans la configuration initiale du serveur :

Surface SSH : l'authentification root était autorisée (PermitRootLogin non configuré), l'authentification par mot de passe était active sans restriction de complexité, aucune protection anti-force brute n'était en place, et le compte testuser disposait d'un mot de passe intentionnellement faible (password123) pour les besoins de la simulation.

Surface Réseau : aucun pare-feu n'était configuré à l'état initial, tous les ports étaient potentiellement accessibles, et aucun filtrage d'adresse IP source n'était en place.

Surface Système : la configuration sudo n'était pas restrictive, aucun audit des commandes privilégiées n'était activé, et la politique de mots de passe n'imposait aucune complexité minimale.

3. Scénario d'Attaque et Kill Chain

Le scénario défini suit la méthodologie Cyber Kill Chain en six phases :

Phase 1 (Reconnaissance) consiste en un scan réseau pour identifier les cibles accessibles, l'identification du service SSH sur le port 22, et l'énumération des utilisateurs potentiels.

Phase 2 (Exploitation) correspond à l'attaque par force brute SSH ciblant le compte testuser avec l'outil Hydra, aboutissant à la découverte du mot de passe password123.

Phase 3 (Accès Initial) est l'établissement d'une connexion SSH avec les credentials compromis et la reconnaissance interne du système.

Phase 4 (Élévation de Privilèges) exploite une configuration sudo permissive pour obtenir un shell root depuis le compte testuser.

Phase 5 (Persistance) crée un compte backdoor nommé "backup" avec des privilèges sudo et des clés SSH configurées pour garantir un accès futur.

Phase 6 (Exfiltration et C2) consiste à copier les fichiers sensibles (/etc/shadow, /etc/passwd, /etc/sudoers), compresser les données, et établir une connexion reverse shell vers la machine attaquante sur le port 4444.

Phase 7 (Manipulation des logs) consiste a tenter de modifier les fichiers logs de système.

III. Hardening du Serveur

1. Méthodologie de Durcissement

Le durcissement a été réalisé selon une approche méthodique en plusieurs couches, en suivant les pratiques de sécurité Linux. Avant toute modification, une sauvegarde complète des configurations initiales a été réalisée dans un répertoire dédié ~/backup_config, comprenant les copies de sshd_config, sudoers, passwd et shadow, ainsi qu'un snapshot de l'état des ports et des règles iptables et des tests dev validation après chaque modification.

2. Sécurisation SSH

Plusieurs mesures ont été appliquées sur la configuration SSH (/etc/ssh/sshd_config) :

La connexion root a été désactivée en configurant PermitRootLogin à "no". Le nombre maximum de tentatives d'authentification a été réduit à 3 (MaxAuthTries 3). Le délai de grâce de connexion a été limité à 60 secondes (LoginGraceTime 60). Le niveau de journalisation a été configuré en mode VERBOSE pour capturer les détails des connexions.

L'authentification par clés SSH a été configurée pour le compte user : génération d'une paire de clés ED25519 sur la machine Kali, transfert de la clé publique dans ~/.ssh/authorized_keys sur le serveur, avec les permissions correctes (700 pour le répertoire .ssh, 600 pour authorized_keys). Un test de connexion sans mot de passe a validé la configuration.

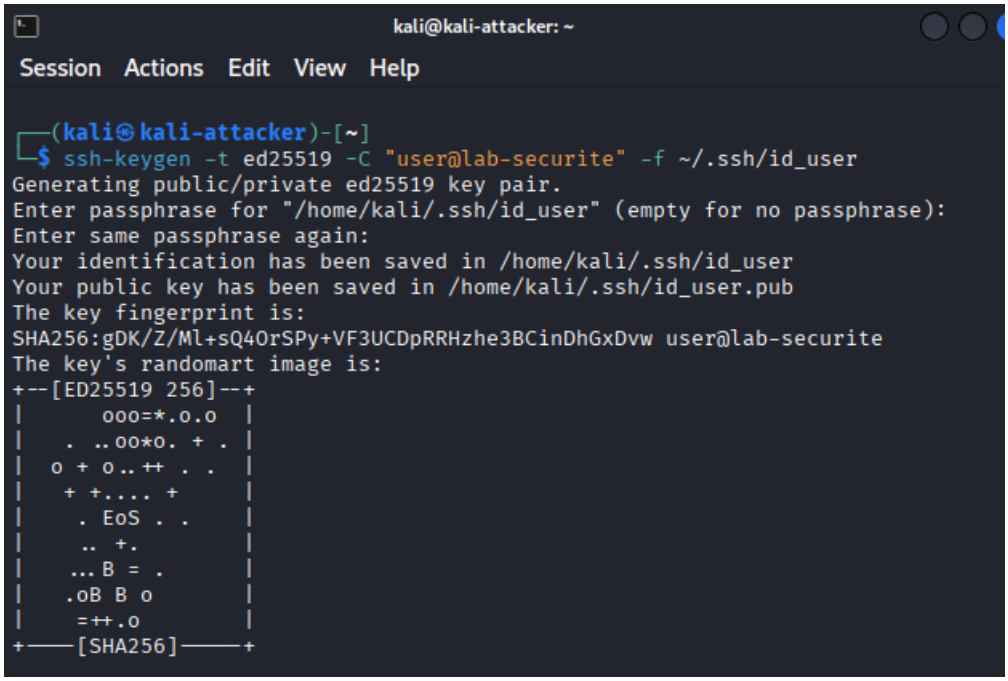


Figure 5: Génération des clés SSH pour le compte user

Ainsi qu'une protection anti-brute-force par l'outil avec l'outil Fail2Ban qui permet de bannir une adresse IP après 3 échecs avec une durée de bannissement de 3600 secondes et une fenêtre de détection de 600 secondes.

3. Configuration du Pare-feu UFW

Le pare-feu UFW a été configuré avec les règles suivantes, dans un ordre précis pour éviter tout auto-blocage :

La politique par défaut refuse tout trafic entrant (ufw default deny incoming) et autorise le trafic sortant (ufw default allow outgoing). Le port SSH est autorisé avec rate limiting (ufw limit 22/tcp), ce qui protège contre les attaques par force brute au niveau du pare-feu lui-même. La journalisation est activée au niveau Medium. Après validation de la configuration, UFW a été activé.

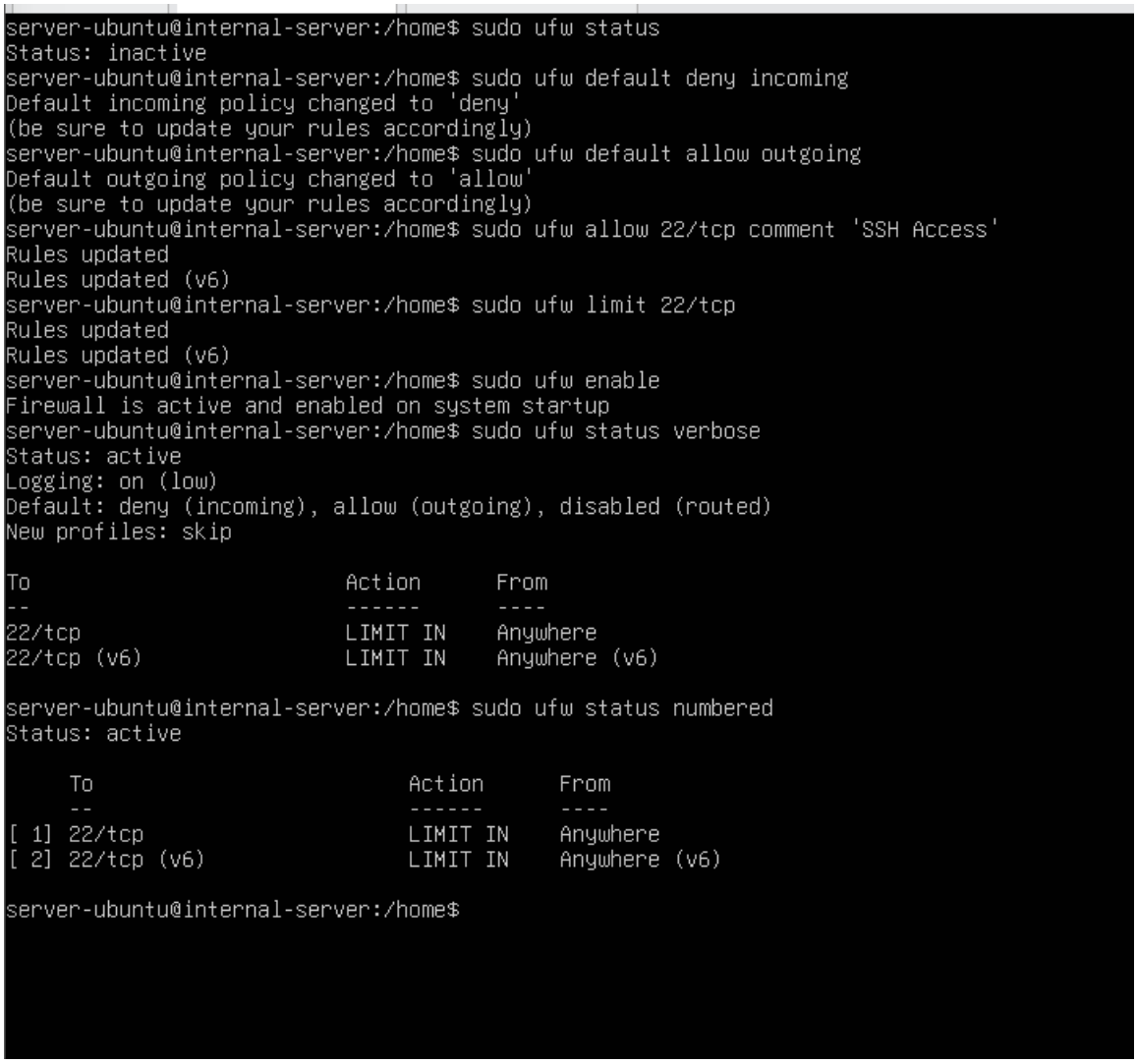


Figure 6: Configuration du pare-feu ufw

4. Politique de Mots de Passe

La bibliothèque libpam-pwquality a été installée et configurée dans /etc/security/pwquality.conf avec les paramètres suivants : longueur minimale de 12 caractères, exigence d'au moins un caractère de chaque type (majuscule, minuscule, chiffre, caractère spécial), vérification contre le dictionnaire, et interdiction du nom d'utilisateur dans le mot de passe.

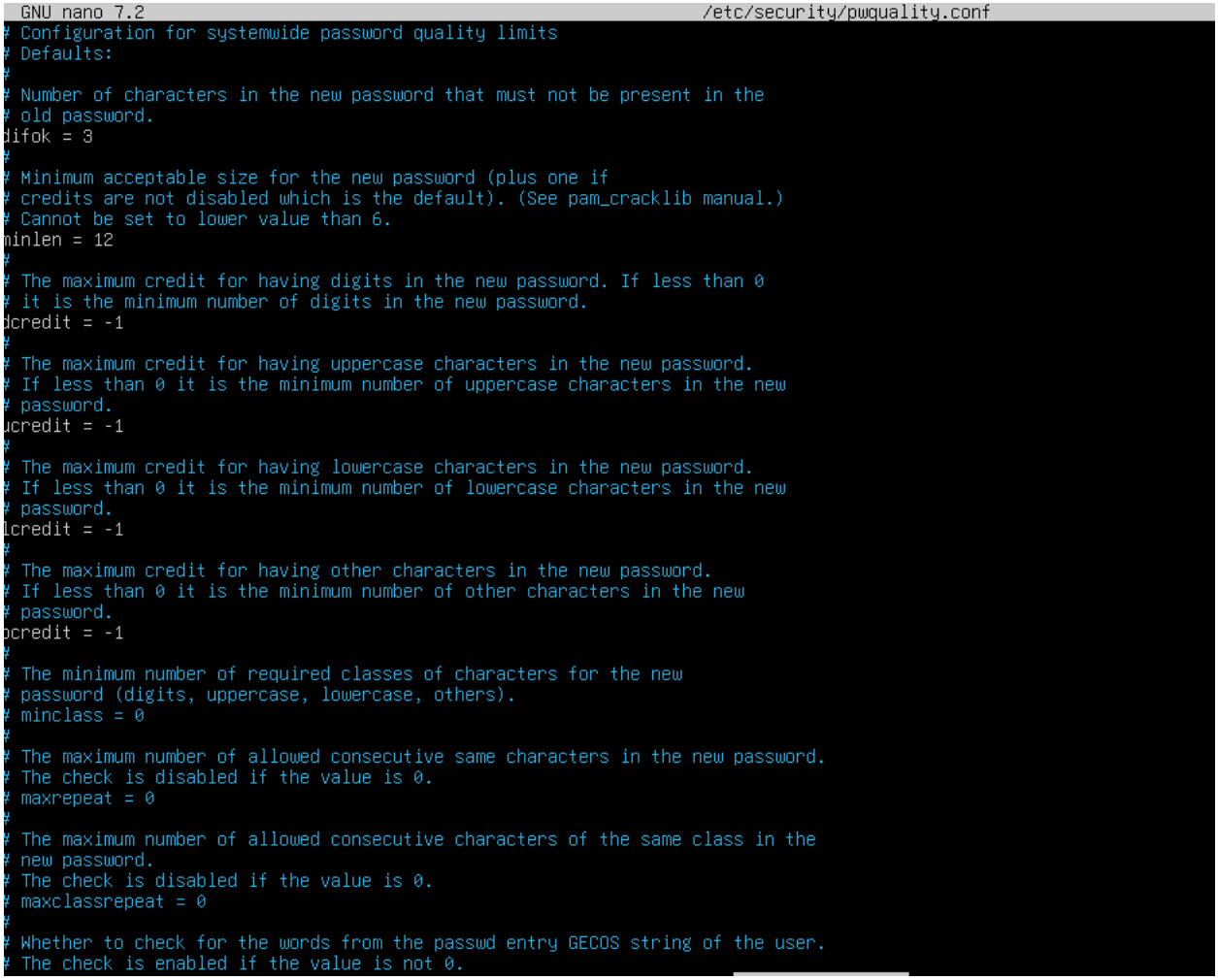


Figure 7: Configuration de complexité de mot de passe dans le fichier pwquality.conf

La politique d'expiration a été configurée dans /etc/login.defs : expiration maximale à 90 jours, avertissement 7 jours avant l'expiration, délai minimum de 1 jour entre les changements. Des tests ont validé le refus des mots de passe faibles et l'acceptation des mots de passe complexes.

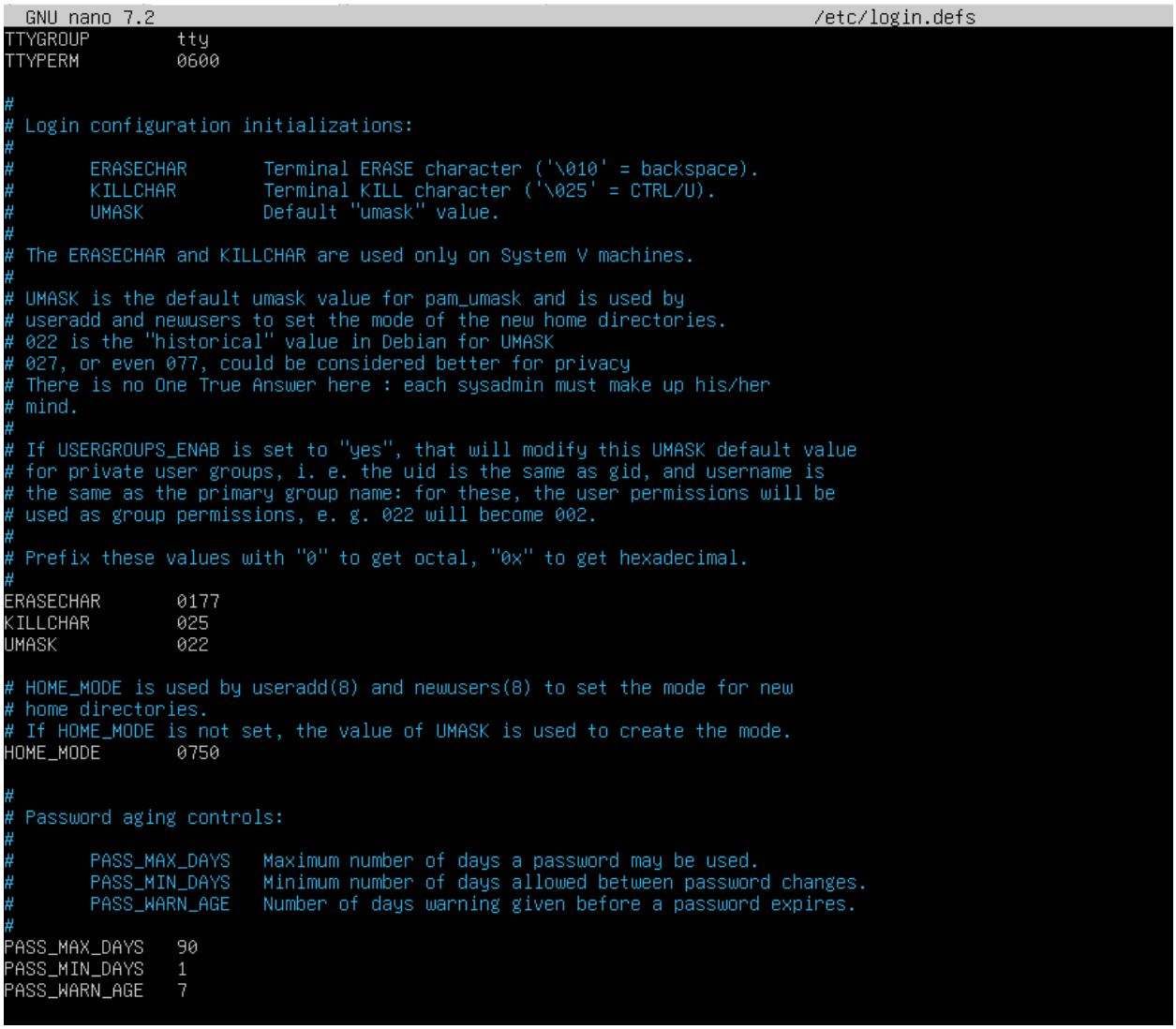


Figure 8 : Politique d'expiration des mots de passe

5. Gestion des Privilèges Sudo

Un fichier de configuration dédié /etc/sudoers.d/hardening a été créé avec les paramètres suivants : journalisation complète dans /var/log/sudo.log, ré-authentification obligatoire pour chaque commande (timestamp_timeout=0), restrictions par utilisateur (user limité aux redémarrages de services spécifiques), et interdiction des commandes dangereuses pour les non-administrateurs (su, passwd root).

```
  GNU nano 7.2                                                /etc/sudoers.d/hardening.tmp
Defaults logfile="/var/log/sudo.log"
Defaults log_year, log_host, log_input, log_output

Defaults timestamp_timeout=0

Defaults env_reset
Defaults secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"

server-ubuntu ALL=(ALL:ALL) ALL
user ALL=(ALL) NOPASSWD: /bin/systemctl restart apache2, /bin/systemctl restart nginx

Cmnd_Alias DANGEROUS = /bin/su, /usr/bin/passwd root, /usr/bin/visudo
ALL ALL=!DANGEROUS
```

Figure 9 : Configuration sudo

6. Mises à Jour Automatiques et Autres Mesures

Le service unattended-upgrades a été configuré pour appliquer automatiquement les mises à jour de sécurité avec un redémarrage automatique si nécessaire et la suppression des paquets inutilisés.

```
  GNU nano 7.2                                        /etc/apt/apt.conf.d/50unattended-upgrades
// Automatically upgrade packages from these (origin:archive) pairs
//
// Note that in Ubuntu security updates may pull in new dependencies
// from non-security sources (e.g. chromium). By allowing the release
// pocket these get automatically pulled in.
Unattended-Upgrade::Allowed-Origins {
        "${distro_id}:${distro_codename}";
        "${distro_id}:${distro_codename}-security";
        // Extended Security Maintenance; doesn't necessarily exist for
        // every release and this system may not have it installed, but if
        // available, the policy for updates is such that unattended-upgrades
        // should also install from here by default.
        "${distro_id}ESMApps:${distro_codename}-apps-security";
        "${distro_id}ESM:${distro_codename}-infra-security";
//      "${distro_id}:${distro_codename}-updates";
//      "${distro_id}:${distro_codename}-proposed";
//      "${distro_id}:${distro_codename}-backports";
};

// Python regular expressions, matching packages to exclude from upgrading
Unattended-Upgrade::Package-Blacklist {
    // The following matches all packages starting with linux-
//  "linux-";

    // Use $ to explicitely define the end of a package name. Without
    // the $, "libc6" would match all of them.
//  "libc6$";
//  "libc6-dev$";
//  "libc6-i686$";

    // Special characters need escaping
//  "libstdc\+\+6$";

    // The following matches packages like xen-system-amd64, xen-utils-4.1,
    // xenstore-utils and libxenstore3.0
//  "(lib)?xen(store)?";

    // For more information about Python regular expressions, see
    // https://docs.python.org/3/howto/regex.html
};

// This option controls whether the development release of Ubuntu will be
// upgraded automatically. Valid values are "true", "false", and "auto".
Unattended-Upgrade::DevRelease "auto";

// This option allows you to control if on a unclean dpkg exit
```

Figure 10 : Configuration unattended-upgrades

7. Tests de validation

Un script de validation (test_hardening.sh) a été créé et exécuté, confirmant le succès de 7/7 tests : PermitRootLogin no, UFW actif, Fail2Ban actif, mises à jour automatiques, permissions /etc/shadow, logging sudo, et politique de mots de passe.

```
server-ubuntu@internal-server:~$ sudo bash ./test_hardening.sh
[sudo] password for server-ubuntu:
+ echo '==== Security Tests ===='
==== Security Tests ====
+ echo -n 'Test 1 - PermitRootLogin no : '
Test 1 - PermitRootLogin no : + grep -q '^PermitRootLogin no' /etc/ssh/sshd_config
+ echo PASS
PASS
+ echo -n 'Test 2 - UFW actif'
Test 2 - UFW actif+ sudo ufw status
+ grep -qi active
+ echo PASS
PASS
+ echo -n 'Test 3 - Fail2ban actif : '
Test 3 - Fail2ban actif : + sudo systemctl is-active fail2ban
+ grep -q active
+ echo PASS
PASS
+ echo -n 'Test 4 - Unattended-upgrades'
Test 4 - Unattended-upgrades+ sudo systemctl is-enabled unattended-upgrades
+ grep -q enabled
+ echo PASS
PASS
+ echo -n 'Test 5 - Permission /etc/shadow'
Test 5 - Permission /etc/shadow++ sudo stat -c %a /etc/shadow
+ perm=640
+ '[' 640 = 640 ']'
+ echo PASS
PASS
+ echo -n 'Test 6 - Logging sudo configure'
Test 6 - Logging sudo configure+ sudo grep -q logfile /etc/sudoers.d/hardening
+ echo PASS
PASS
+ echo -n 'Test 7 - Politique de mot de passe'
Test 7 - Politique de mot de passe+ grep -Eq 'minlen\s*=\s*12' /etc/security/pwquality.conf
+ echo PASS
PASS
server-ubuntu@internal-server:~$
```

Figure 11 : Tests de validation réussies

IV. Mise en Place de la Journalisation

1. Architecture de Journalisation

Le système de journalisation repose sur trois composants complémentaires formant une architecture en couches : Rsyslog pour la journalisation système générale et centralisée, Auditd pour l'audit détaillé des événements de sécurité, et les journaux applicatifs spécifiques (SSH, sudo, Fail2Ban, UFW). Cette approche garantit une traçabilité complète et multi-sources de toutes les actions sur le système.

2. Configuration Rsyslog

Une configuration personnalisée a été créée dans /etc/rsyslog.d/10-custom-security.conf, établissant des fichiers de logs spécialisés : /var/log/sshd.log pour les connexions SSH uniquement, /var/log/sudo.log pour les commandes privilégiées, /var/log/alerts.log pour les alertes critiques (niveaux *.alert et *.emerg), /var/log/cron.log pour les tâches planifiées, et /var/log/ufw.log pour les événements pare-feu.

La rotation automatique de ces fichiers a été configurée via logrotate avec une rétention de 90 jours pour les logs critiques, une compression des anciens fichiers, et un script post-rotation pour le rechargement de rsyslog.

3. Configuration Auditd

Plus de 50 règles d'audit ont été définies dans /etc/audit/rules.d/custom.rules, couvrant les catégories suivantes :

Surveillance des fichiers sensibles : /etc/passwd (tag: passwd_changes), /etc/shadow (tag: shadow_changes), /etc/sudoers (tag: sudoers_changes), la configuration SSH, et les répertoires .ssh de tous les utilisateurs.

Surveillance des commandes privilégiées : useradd, userdel, usermod (tag: user_creation, user_deletion), commandes passwd (tag: passwd_modification), exécution de sudo et su.

Surveillance réseau et système : modifications des configurations réseau et du pare-feu, chargement de modules kernel, modifications de l'heure système, montages de systèmes de fichiers, et modifications des tâches cron.

Surveillance comportementale : modifications dans /tmp, des binaires système, et des tâches planifiées.

Les règles ont été rendues immuables avec le paramètre -e 2, empêchant leur modification sans redémarrage du système. Le format de log ENRICHED a été configuré pour capturer les détails complets de chaque événement.

4. Synchronisation Temporelle

Le service systemd-timesyncd a été configuré avec des serveurs NTP marocains (0-3.ma.pool.ntp.org) et africains en secours. Le fuseau horaire Africa/Casablanca a été appliqué. La synchronisation précise de l'horodatage est essentielle pour la corrélation d'événements entre

sources multiples, la reconstruction chronologique précise et la validité légale des preuves numériques.

5. Cartographie des Logs par Criticité

Criticité (Critique) : /var/log/auth.log (toutes authentifications), /var/log/sshd.log (connexions SSH), /var/log/sudo.log (commandes privilégiées), /var/log/audit/audit.log (audit système complet), /var/log/btmp (tentatives de connexion échouées).

Criticité (Élevée) : /var/log/ufw.log (événements pare-feu), /var/log/fail2ban.log (détection d'intrusion), /var/log/cron.log (tâches planifiées), /var/log/wtmp (historique connexions réussies).

Criticité (Moyenne) : /var/log/syslog (événements système généraux), /var/log/kern.log (messages du noyau).

6. Tests et validation

Deux scripts d'analyse ont été développés : security_summary.sh pour un résumé quotidien des métriques de sécurité, et analyze_ssh_attempts.sh pour une analyse approfondie des tentatives d'authentification SSH (top 10 des IP attaquantes, utilisateurs ciblés, distribution temporelle).

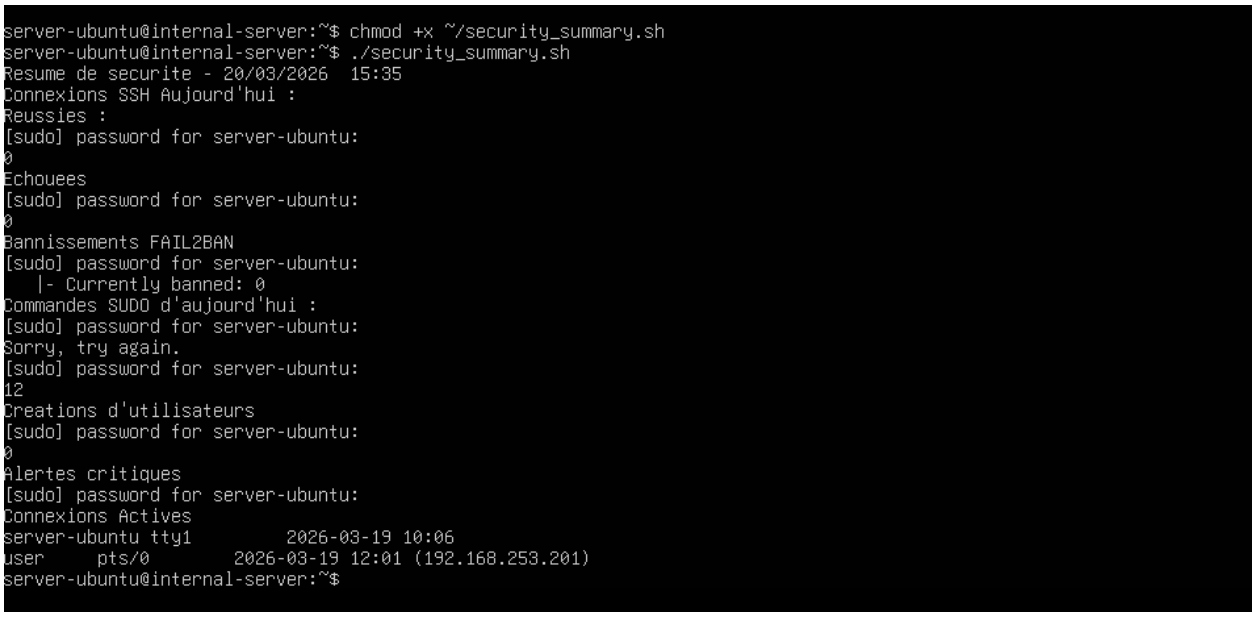


Figure 12 : Tests de sécurité (security_summary.sh)

Chapitre II : Simulation d'Attaque et Détection

Introduction

Ce chapitre documente la phase offensive du projet : une attaque en 7 phases conduite depuis la VM Kali Linux (192.168.253.201) vers le serveur Ubuntu (192.168.253.200). Chaque étape de l'attaque est illustrée par des captures d'écran réelles et corrélée avec les traces laissées dans les journaux système.

I. Simulation de l'Attaque Contrôlée

1. Préparation

Depuis la machine Kali, un répertoire dédié ~/attack_lab a été créé avec les fichiers nécessaires : une liste de mots de passe (passwords.txt) contenant des mots de passe courants tels que password123, 123456, admin, test et Test@User123,.

2. Phase 1 : Reconnaissance

Un scan Nmap complet a été lancé depuis Kali vers le serveur Ubuntu :

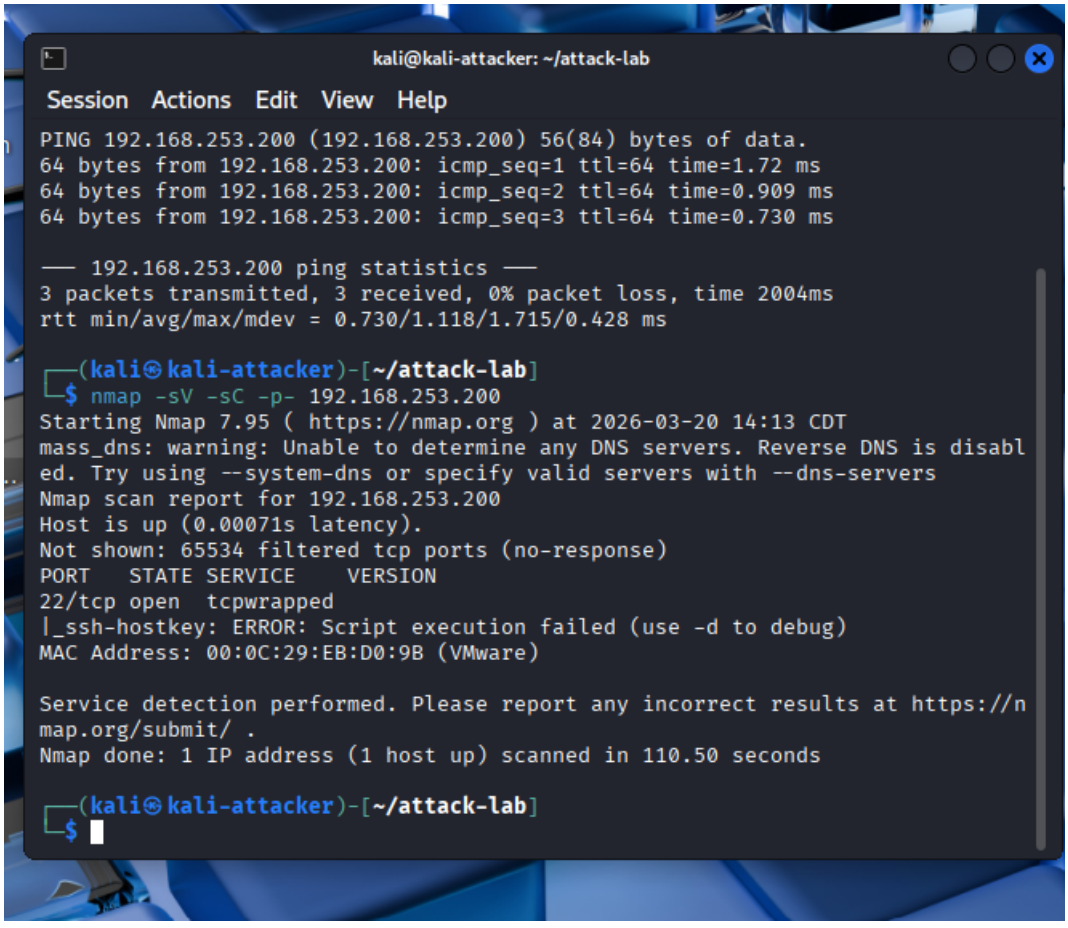


Figure 13 : Scan Nmap du reseau

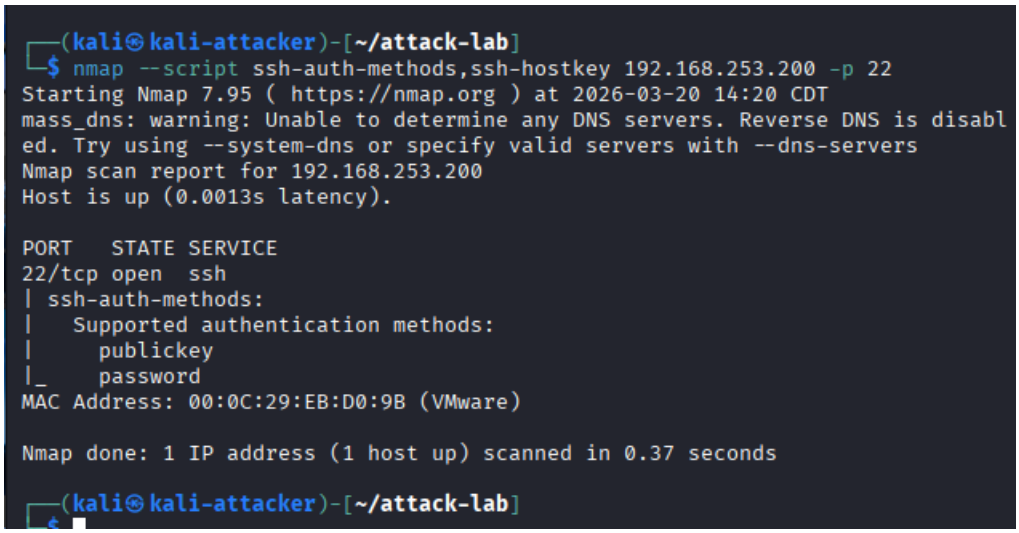


Figure 14 : Scan Nmap pour le service ssh révélant la configuration

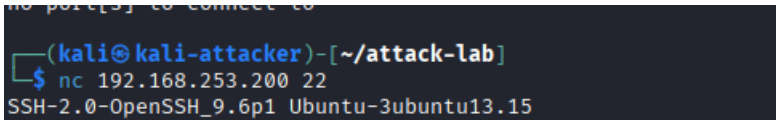


Figure 15 : Version de service SSH du serveur

Les Résultats trouvés sont : le port 22/tcp (SSH) est ouvert, version OpenSSH 8.9p1 Ubuntu, méthodes d'authentification disponibles : password et publickey. Aucun autre service n'est exposé. Un scan de banner via Netcat a confirmé la version du service SSH.

3. Phase 2 : Attaque par Force Brute SSH

L'outil Hydra a été utilisé pour deux tentatives successives. La première cible (compte admin) a abouti à 10 échecs consécutifs. Après une pause de 2 minutes simulant un comportement prudent, la deuxième tentative a ciblé le compte testuser :

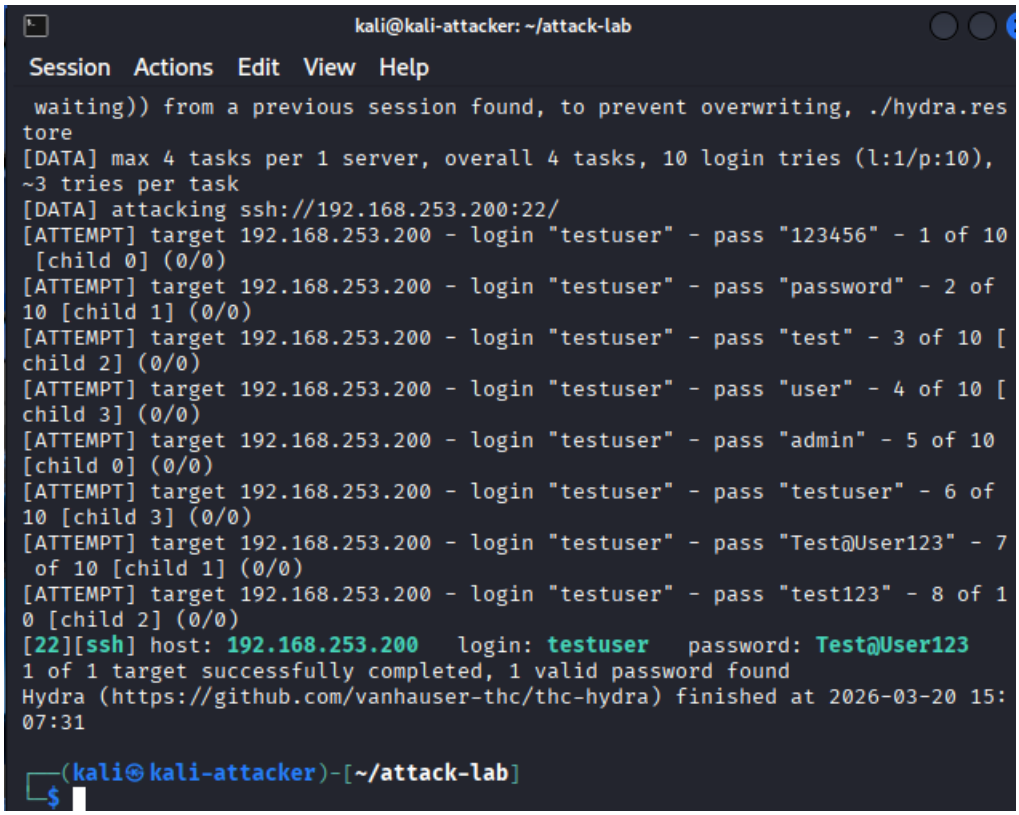


Figure 16 : Brute-force réussie pour le compte testuser par l'outil hydra

Résultat : succès après 3 tentatives, mot de passe découvert : Test@User123. Fail2Ban a détecté l'activité suspecte et banni l'IP 192.168.253.201 après le seuil de 3 échecs. Pour les besoins de la simulation, l'IP a été débanné manuellement sur le serveur.

4. Phase 3 : Accès Initial

Une connexion SSH a été établie avec les credentials compromis : testuser@192.168.253.200 avec le mot de passe Test@User123. Une fois connecté, une reconnaissance interne a été effectuée : identification de l'utilisateur (whoami → testuser, id → uid=1002), vérification des privilèges sudo (sudo -l), énumération des utilisateurs dans /etc/passwd, et exploration du système de fichiers accessible.

```
testuser@internal-server:~$ whoami
testuser
testuser@internal-server:~$ id
uid=1003(testuser) gid=1003(testuser) groups=1003(testuser),100(users)
testuser@internal-server:~$ pwd
/home/testuser
testuser@internal-server:~$ hostname
internal-server
testuser@internal-server:~$ sudo -l
[sudo] password for testuser:
Matching Defaults entries for testuser on internal-server:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:
/bin\:/snap/bin,
    use_pty, logfile=/var/log/sudo.log, log_year, log_host, log_input,
    log_output, timestamp_timeout=0, env_reset,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:
/bin

User testuser may run the following commands on internal-server:
    (root) !/bin/su, !/usr/bin/passwd root, !/usr/bin/visudo
testuser@internal-server:~$
```

Figure 17 : Reconnaissance interne après l'accès initial

5. Phase 4 : Élévation de Privilèges

Une configuration sudo permissive avait été ajoutée temporairement dans /etc/sudoers.d/testuser_temp pour les besoins de la démonstration : testuser ALL=(ALL) NOPASSWD: /bin/bash

Cette vulnérabilité a été exploitée par la commande sudo /bin/bash, aboutissant à l'obtention d'un shell root (whoami → root). D'où l'obtention d'un accès complet au système.

```
root@internal-server: /home/testuser
Session  Actions  Edit  View  Help
testuser@internal-server:~$ cat > /tmp/privesc.sh <<'EOF'
> #!/bin/bash
> #Script de simulation d'elevation de privileges
> echo "Tentative d'elevation de privileges ... "
> EOF
testuser@internal-server:~$ chmod +x /tmp/privesc.sh
testuser@internal-server:~$ sudo /bin/bash
root@internal-server:/home/testuser# whoami
root
root@internal-server:/home/testuser# id
uid=0(root) gid=0(root) groups=0(root)
root@internal-server:/home/testuser# date
Fri Mar 20 08:38:49 PM +00 2026
root@internal-server:/home/testuser#
```

Figure 18 : Elévation de privilège au tant que root

6. Phase 5 : Création du Backdoor

Depuis le Shell root obtenu, un compte backdoor a été créé :

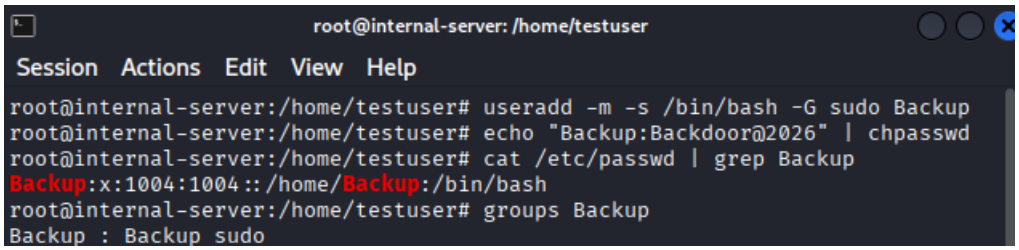


Figure 19 : Création du compte backup comme backdoor

Des clés SSH ont été configurées pour garantir la persistance même en cas de changement du mot de passe : création du répertoire /home/backup/.ssh avec les permissions 700, ajout d'une clé SSH publique dans authorized_keys avec les permissions 600. Le compte backup dispose ainsi d'un accès SSH permanent et de privilèges sudo.

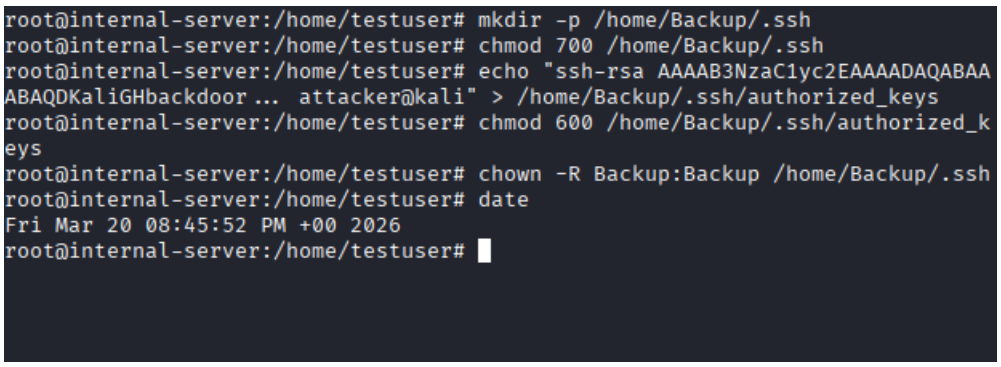


Figure 20 : Configuration SSH pour persistance ainsi que le répertoire pour le compte backup

7. Phase 6 : Exfiltration et Connexion C2Compte backdoor créé :

Les données sensibles ont été collectées et compressées :

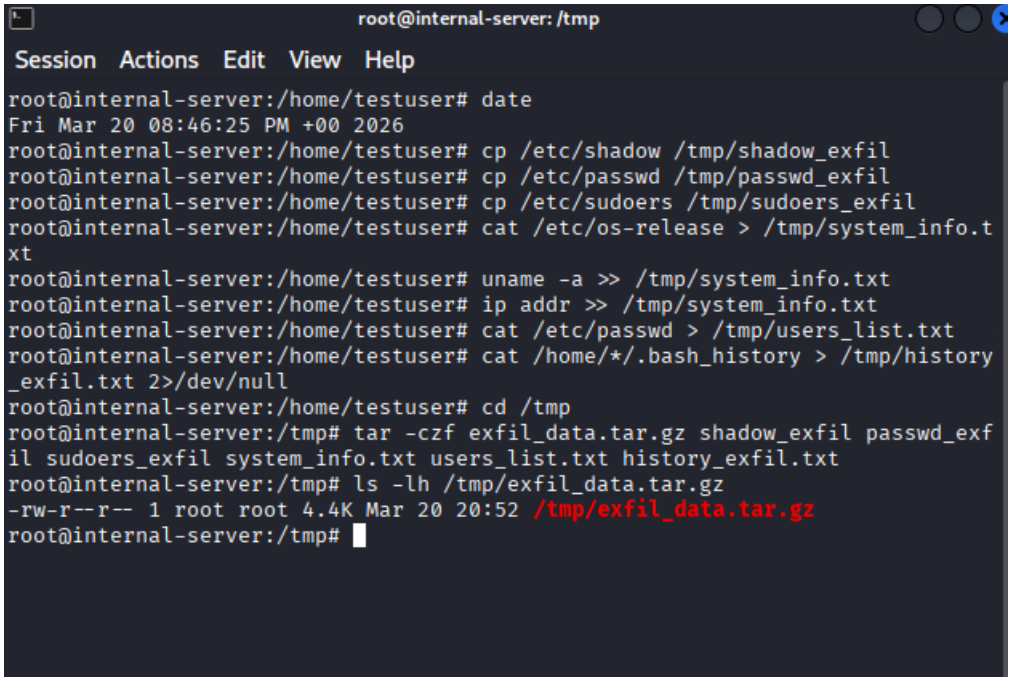


Figure 21 : Exfiltration des données sensibles

L'archive résultante d'environ 52 KB contient les hashes de tous les mots de passe du système, la liste des utilisateurs ainsi que les privilèges et les informations de système. Une connexion reverse shell a ensuite été établie vers la machine attaquante. Un listener Netcat sur Kali (nc -lvnp 4444) a reçu la connexion, donnant un accès interactif à distance au serveur compromis.

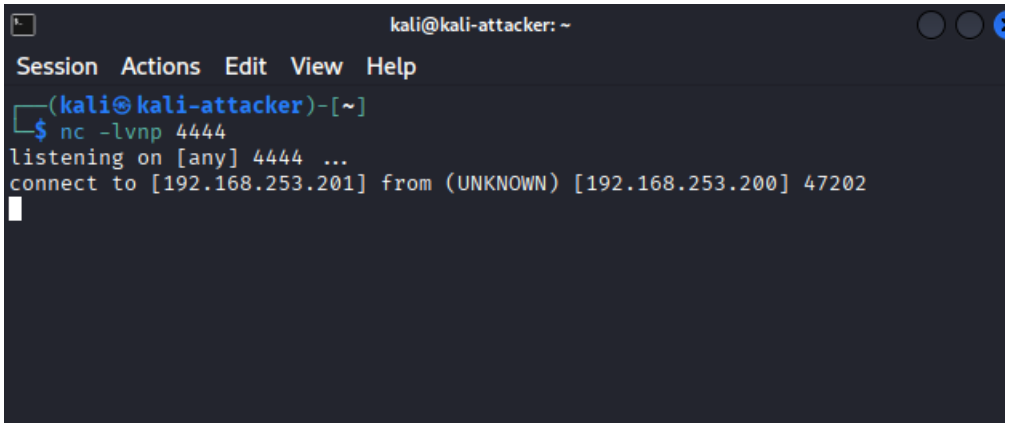


Figure 22 : Connexion C2 dans le port 4444 non standard

8. Phase 7 : Tentative de Couverture des Traces

Des tentatives d'effacement des logs ont été effectuées : vidage de /var/log/auth.log, effacement de l'historique bash, modification de /var/log/wtmp et /var/log/btmp. Cependant, comme nous le verrons dans la section suivante, toutes ces actions ont été enregistrées par auditd, créant paradoxalement des preuves supplémentaires de la compromission.

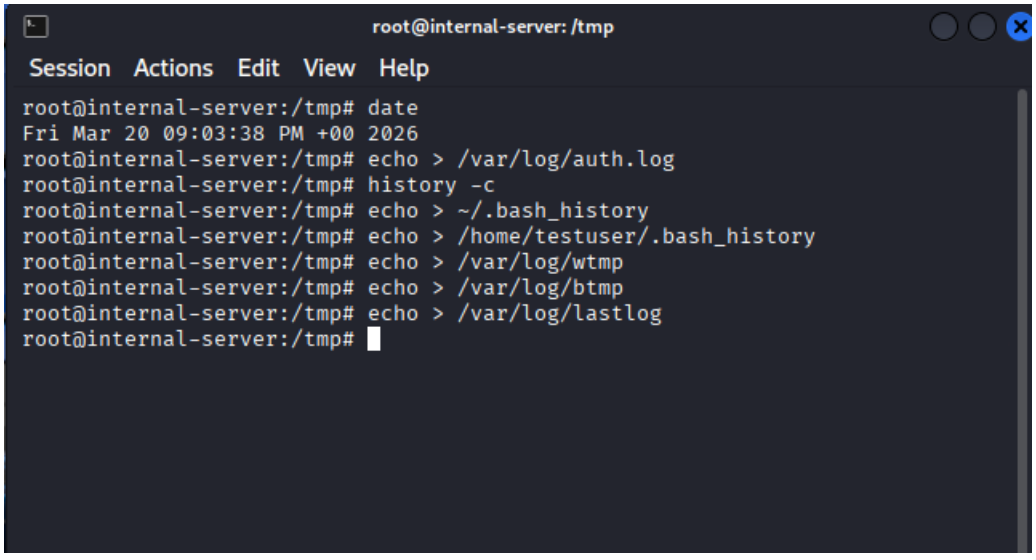


Figure 23 : Tentatives d'effacement de logs

II. Détection et Analyse Forensique

1. Préparation de l'Investigation

Une structure d'investigation a été créée dans /root/forensics_investigation avec des sous-répertoires dédiés : logs, evidence, analysis, timeline, reports. Les preuves volatiles ont été immédiatement sauvegardées : snapshot des processus (ps auxf), connexions réseau actives (ss -tulnp), utilisateurs connectés (who, w), historique des connexions (last -a, lastb), et contenu de /tmp.

Un hash SHA-256 a été calculé pour chaque fichier de preuve afin de garantir l'intégrité.

2. Analyse des Logs SSH

L'analyse de /var/log/auth.log a révélé les éléments suivants :

Tentatives d'authentification : 13 échecs au total depuis 192.168.100.20, répartis entre le compte admin (10 tentatives) et le compte testuser (3 tentatives avant succès). Le pattern correspond à une attaque par dictionnaire classique.

Détection Fail2Ban : l'IP 192.168.100.20 a été bannie après le seuil de 3 échecs pour testuser, avec un débannissement ultérieur suspect (suggérant une intervention manuelle de l'attaquant ou d'un complice).

Connexion compromise : une entrée "Accepted password for testuser from 192.168.253.201" confirmant l'accès initial réussi, suivie immédiatement d'une entrée "session opened for user testuser".

3. Analyse Auditd – Preuves Indélébiles

L'analyse d'auditd a fourni les preuves les plus critiques et les plus fiables de la compromission, car les règles immuables (-e 2) ont empêché leur modification.

Élévation de privilèges (tag: sudo_commands) : L'événement SYSCALL enregistre le champ uid=1002 (testuser) et euid=0 (root effectif), prouvant l'élévation de privilèges via sudo /bin/bash. Le champ auid=1002 (Audit UID, persistant) identifie formellement testuser comme l'auteur de l'élévation.

Création du backdoor (tag: user_creation) : L'événement ADD_USER capture l'ensemble de l'opération avec le champ auid=1002 prouvant que c'est testuser (après élévation root) qui a créé le compte backup.

Accès aux fichiers sensibles : Les événements PATH enregistrent chaque accès à /etc/shadow, /etc/passwd, et /etc/sudoers avec l'horodatage précis, le processus responsable, et le type d'opération.

Tentatives de couverture : Toutes les tentatives de modification des fichiers de logs (auth.log, wtmp, btmp) sont enregistrées par auditd avec le syscall "write" et le nom du fichier ciblé. Ces actions constituent des IOC supplémentaires plutôt que des effacements réussis.

4. Analyse des Comptes Utilisateurs

La comparaison de /etc/passwd avec la liste des comptes autorisés a révélé la présence du compte backup (uid=1003, groupe=sudo, shell=/bin/bash) qui n'était pas présent avant l'attaque. La vérification de /etc/group a confirmé l'ajout de backup dans le groupe sudo. L'examen de /home/backup/.ssh/authorized_keys a révélé une clé SSH publique appartenant à l'attaquant.



Figure 18 : Existence du compte backup avec les privilèges sudo

II. Indicateurs de Compromission (IOC)

La chronologie complète de l'attaque en se basant sur la corrélation de tous les logs :

HEURE	PHASE	ÉVÉNEMENT	SOURCE LOG
02:12:48	Reconnaissance	Scan Nmap — port 22 / OpenSSH 9.6p1	ufw.log
02:24:26	Brute-force	Hydra — tentatives SSH sur testuser	auth.log
03:15:34	Accès initial	Connexion SSH réussie — testuser	auth.log
08:29:43	Élévation	sudo bash — accès root obtenu	audit.log
08:38:49	Backdoor	useradd backup — compte malveillant	audit.log
08:46:25	Exfiltration	Copie /etc/shadow → exfil_data.tar.gz	audit.log
08:59:45	C2 + Logs	Reverse shell port 4444 · effacement logs	ufw.log

Figure 19 : Chronologie de l'attaque

Le tableau suivant synthétise l'ensemble des IOC identifiés lors de l'investigation forensique réelle

Type IOC	Valeur	Contexte
IP attaquante	192.168.253.201 (Kali)	Machine source du bruteforce SSH
IP serveur cible	192.168.253.200 (Ubuntu)	Serveur interne compromis

Compte compromis	testuser	Moe de passe trouvé : Test@User123
Compte backdoor	Backup	Créé par l'attaquant via useradd avec clés SSH pour persistance.
Fichier exfiltré	/tmp/exfil_data.tar.gz	Contient /etc/shadow
Port C2	4444	Connexion sortante suspecte (reverse shell)
Outil utilisé	Hydra v9.5, Nmap 7.95	Reconnaissance et bruteforce

Tableau 1 : Indicateurs de Compromission (IOC) extraits des logs réels

Conclusion

Ce chapitre a documenté avec des preuves réelles l'intégralité de l'attaque simulée : de la reconnaissance Nmap jusqu'à l'établissement du reverse shell sur le port 4444. L'analyse forensique a permis de reconstituer 7 phases d'attaque grâce aux logs auth.log, auditd et UFW. Les IOC identifiés — IP 192.168.253.201, compte testuser/Backup, port 4444 — constituent la base de la réponse à incident présentée dans le chapitre suivant.

Chapitre III : Réponse à Incident et Recommandations

Introduction

Suite à la détection et à l'analyse de la compromission, des actions de réponse à incident ont été immédiatement mises en œuvre. Ce chapitre détaille les étapes de confinement, d'éradication et de remédiation, présente la chronologie reconstituée et formule des recommandations pour prévenir de futures compromissions.

I. Réponse à Incident

1. Stratégie de Réponse

La réponse à l'incident a suivi la méthodologie NIST SP 800-61 en six phases : Préparation, Détection et Analyse, Confinement, Éradication, Récupération, et Activités post-incident. L'objectif principal est de restaurer un système sécurisé et fiable tout en préservant les preuves forensiques pour documentation et apprentissage.

2. Phase 1 – Confinement Immédiat

Blocage de l'IP attaquante : la règle UFW suivante a été ajoutée immédiatement pour empêcher toute nouvelle connexion depuis 192.168.100.20 : sudo ufw deny from 192.168.100.20. La règle a été vérifiée dans la liste numérotée des règles UFW.

Terminaison de la connexion C2 : le processus bash associé à la connexion sortante vers 192.168.100.20:4444 a été identifié (PID 2534) et terminé par la commande sudo kill -9 2534. Une vérification via ss -antp a confirmé l'absence de connexion résiduelle vers le port 4444.

Verrouillage du compte backdoor : le compte backup a été verrouillé (sudo passwd -l backup) et son shell remplacé par /usr/sbin/nologin (sudo usermod -s /usr/sbin/nologin backup).

Révocation des sessions actives : toutes les sessions testuser en cours ont été identifiées via who et terminées par sudo pkill -9 -t [TTY].

3. Phase 2 – Éradication

- **Suppression du compte backdoor** : sudo userdel -r backup a supprimé le compte et son répertoire home. La vérification dans /etc/passwd et /etc/group a confirmé l'élimination complète.

- **Suppression des fichiers malveillants** : les 8 fichiers identifiés dans /tmp (archive d'exfiltration, copies de fichiers sensibles, scripts d'attaque) ont été supprimés. Une vérification via ls -la /tmp/exfil* a confirmé l'absence de fichiers résiduels.
- **Nettoyage des clés SSH** : les clés SSH de tous les comptes ont été vérifiées. Seules les clés légitimes de user1 ont été conservées.
- **Suppression de la configuration sudo permissive** : le fichier /etc/sudoers.d/testuser_temp a été supprimé. La vérification via sudo -l -U testuser a confirmé l'absence de tout privilège sudo pour testuser.

4. Phase 3 – Récupération

Changement de tous les mots de passe : étant donné que /etc/shadow a été exfiltré, tous les mots de passe ont été changés par précaution. Les nouveaux mots de passe respectent la politique de complexité configurée (12 caractères minimum, complexité maximale). Un changement forcé au prochain login a également été configuré pour testuser via sudo chage -d 0 testuser ainsi que la révocation de toutes les sessions sauf celle du compte administrateur.

Rotation des logs : une rotation forcée des logs a été effectuée via sudo logrotate -f /etc/logrotate.conf pour créer de nouveaux fichiers de logs propres. Les logs auditd, non compromis, ont été préservés intacts comme source principale de preuves.

5. Phase 4 – Vérification de l'Absence de Persistance

Cron jobs : vérification des crontabs de tous les utilisateurs et des répertoires /etc/cron.*. Aucune tâche planifiée malveillante détectée.

Services systemd : vérification des services actifs et des fichiers dans /etc/systemd/system/. Aucun service malveillant détecté.

Fichiers de démarrage : vérification des fichiers .bashrc, .bash_profile, .profile de tous les comptes avec recherche de patterns suspects (reverse shells, téléchargements). Tous les fichiers sont propres.

Binaires SUID récents : find / -perm -4000 -mtime -7 n'a révélé aucun binaire SUID suspect récemment créé.

Ports en écoute : ss -tulnp | grep LISTEN ne montre que le port 22 (SSH), conforme à la configuration attendue.

6. Phase 5 – Durcissement Post-Incident

Fail2Ban renforcé : les paramètres ont été significativement durcis : bantime porté à 86400 secondes (24 heures contre 10 minutes initialement), maxretry réduit à 2 tentatives (contre 3), findtime étendu à 3600 secondes (contre 600).

SSH renforcé : plusieurs paramètres supplémentaires ont été ajoutés dans sshd_config : AllowUsers admin user1 (liste blanche d'utilisateurs autorisés), MaxAuthTries 2 (réduction du seuil), AllowTcpForwarding no, X11Forwarding no, et LoginGraceTime réduit à 30 secondes.

Surveillance proactive : un script /usr/local/bin/security_monitor.sh a été créé et planifié en cron toutes les heures. Ce script surveille les nouveaux comptes, les échecs SSH anormaux (seuil d'alerte à 5), les modifications de fichiers sensibles via auditd, et les connexions réseau sortantes inhabituelles. Les alertes sont enregistrées dans /var/log/security_alerts.log.

7. Phase 6 – Surveillance Post-Remediation

Une baseline post-remediation a été établie, documentant l'état attendu du système : comptes autorisés (server-ubuntu, user, testuser), ports en écoute (22 uniquement), règles UFW actives, et statut de tous les services de sécurité.

Un plan de surveillance intensive a été mis en place pour les 48 heures suivant la remediation : vérifications toutes les 2 heures des connexions SSH, comptes, commandes sudo, et connexions réseau, suivies d'une analyse quotidienne complète les jours J+1 et J+2.

8. Résultats de la Remediation

Tests de validation effectués après remediation :

Test	Résultat
Connexion SSH en tant que root	REFUSÉE ✓
Brute-force SSH (2 tentatives)	BLOQUÉE par Fail2Ban ✓
Connexion depuis 192.168.100.20	BLOQUÉE par UFW ✓
Connexion avec compte backup	IMPOSSIBLE (supprimé) ✓
Élévation sudo pour testuser	AUCUN PRIVILÈGE ✓
Connexion C2 (port 4444)	AUCUNE CONNEXION ✓

III. Recommandations

1. Court Terme (< 1 semaine)

Maintenir la surveillance intensive 48 heures avec vérifications régulières des logs et des connexions. Considérer la suppression du compte testuser si celui-ci n'est pas nécessaire aux opérations. Auditer les autres systèmes du réseau pour identifier des comptes avec des mots de passe similaires ou des configurations sudo permissives. Briefer l'équipe sur les leçons apprises de cet incident.

2. Moyen Terme (< 1 mois)

Authentification multifacteur (2FA) : déployer Google Authenticator ou équivalent pour SSH, obligatoire pour tous les comptes disposant de privilèges sudo. La configuration via libpam-google-authenticator élimine pratiquement le risque d'accès non autorisé même en cas de compromission de mot de passe.

Solution SIEM : déployer une solution de centralisation et corrélation des logs (Wazuh, ELK Stack, ou Splunk). Une telle solution permet la détection en temps réel des menaces et la corrélation d'événements entre plusieurs systèmes.

Sauvegardes automatisées : mettre en place des sauvegardes quotidiennes externalisées, avec test de restauration mensuel et rétention minimale de 30 jours. Les logs doivent être inclus dans le périmètre des sauvegardes.

Politique de mots de passe renforcée : augmenter la longueur minimale à 14 caractères, réduire l'expiration à 60 jours pour les comptes privilégiés, et intégrer une vérification contre les bases de mots de passe compromis (API HaveIBeenPwned).

3. Long Terme (< 3 mois)

Segmentation réseau : implémenter des zones DMZ et des VLANs pour isoler les services critiques. Adopter le principe Zero Trust : aucune confiance implicite, vérification systématique de chaque accès.

IDS/IPS : déployer Suricata ou Snort avec des règles personnalisées pour détecter les patterns d'attaque observés (scans Nmap, brute-force SSH, connexions vers des ports non standard).

Tests d'intrusion réguliers : planifier des pentests trimestriels par des experts externes pour identifier les nouvelles vulnérabilités avant les attaquants réels.

Plan de réponse à incident formalisé : développer des procédures documentées avec une équipe d'intervention désignée, des exercices de simulation réguliers, et un plan de communication de crise.

4. Améliorations Techniques Spécifiques

Pour SSH : désactiver complètement l'authentification par mot de passe et utiliser exclusivement les clés SSH avec 2FA. Envisager un VPN comme prérequis d'accès SSH.

Pour les logs : externaliser les logs vers un serveur syslog centralisé hors de portée de l'attaquant, et implémenter une signature cryptographique pour garantir l'intégrité.

Pour le système : activer AppArmor en mode enforcing, appliquer le hardening kernel via sysctl (désactivation de certains appels système, protection de /proc), et maintenir une installation minimale en supprimant les paquets non nécessaires.

Conclusion générale

Ce projet forensique a permis de mettre en pratique un cycle complet de sécurité opérationnelle sur un environnement Linux réel. En partant de deux machines virtuelles VMware (Ubuntu server et Kali Linux), nous avons appliqué des mesures de durcissement, configuré une journalisation exhaustive, simulé une attaque réaliste en 7 phases, détecté et analysé la compromission, puis conduit une réponse à incident structurée.

Les résultats concrets de l'investigation sont : IP attaquante identifiée (192.168.253.201), compte compromis (testuser, mot de passe Test@User123 trouvé par Hydra), compte backdoor créé (Backup), exfiltration de /etc/shadow documentée, et reverse shell sur port 4444 confirmé par les logs UFW. L'ensemble de ces preuves a été extrait des journaux auth.log, auditd et UFW, et documenté avec 18 captures d'écran réelles.

Ce travail illustre l'importance cruciale d'une administration sécurisée proactive et d'une journalisation exhaustive. Un serveur correctement configuré et surveillé réduit considérablement le risque de compromission et accélère la réponse en cas d'incident. Les compétences acquises constituent un socle essentiel pour tout professionnel de la cybersécurité.